DETAILED PROCEDURE

Setting Up Web Tunneling and Webhooking

ngrok authentication, HTTP forwarding, and the n8n WEBHOOK_URL environment variable

Course	Class	Audience	n8n default port
Tools Integration with n8n	Class three	Local n8n on Windows / macOS / Linux	**5678**

This document expands the class notes and the attached webhooking checklist into a complete, step-by-step procedure. Follow the sections in order: authenticate ngrok, open a public HTTPS tunnel to your local n8n instance, then tell n8n to advertise that public URL as its webhook base.

1. Purpose and how the pieces fit together

When n8n runs on your laptop it listens only on localhost (typically http://localhost:5678). External services such as Telegram, payment gateways, or other SaaS products cannot reach localhost. ngrok solves that by creating a secure public HTTPS URL that forwards every request into your local n8n process.

n8n then needs to know that public URL so that Webhook nodes, Telegram Trigger nodes, and similar triggers register the correct callback address. That is the role of the WEBHOOK_URL environment variable.

1.1 Architecture

External service	Public URL	ngrok cloud	Local tunnel	n8n workflow
Telegram, payment gateway, or any HTTP client	`https://xxxx.ngrok-free.app`	Encrypted reverse proxy	`http://localhost:5678`	Webhook / Telegram Trigger node

1.2 Why ngrok is used

- Exposes a local web server to the public internet through a secure tunnel.
- Encrypts traffic so data is protected in transit.
- Removes the need for a public IP address or router port forwarding.
- Gives you a shareable HTTPS URL that points at your local n8n.

2. Prerequisites

- A free ngrok account at https://dashboard.ngrok.com (Log in or Sign up).
- ngrok agent installed on the same PC that will run n8n.
- n8n already installed locally (Windows: n8n.cmd; other platforms: the n8n CLI).
- Command Prompt or PowerShell on Windows; Terminal on macOS / Linux.
- If you will test with Telegram: a bot token from @BotFather.

Important about free ngrok URLs

On the free plan the public hostname (for example https://fddf8e263fe3.ngrok-free.app) usually changes every time you start a new tunnel session. After each restart you must copy the new URL and update WEBHOOK_URL before launching n8n again.

Do not treat sample hostnames from class notes as your own. Always copy the URL that your current ngrok session prints.

3. Install ngrok on the local machine

From the ngrok dashboard open Getting Started → Setup & Installation. Choose your operating system (Windows, macOS, or Linux) and follow the official installer. On Windows you can use Microsoft Store, WinGet, or a direct download.

Typical Windows options

```
winget install Ngrok.Ngrok

# or download the zip from the dashboard and extract ngrok.exe
# to a folder such as D:\ngrok  then add that folder to PATH
```

Confirm the agent is on your PATH:

```
ngrok version
```

Tip

Keep the ngrok terminal window open for the whole lab. Closing it tears down the tunnel and every webhook that used that public URL will fail.

4. Set up authentication (ngrok authtoken)

The authtoken binds the local agent to your ngrok account. You only need to do this once per machine.

4.1 Copy the token from the dashboard

1. Open `https://dashboard.ngrok.com/get-started/your-authtoken` (Getting Started → Your Authtoken).
2. Copy the personal Authtoken. Treat it like a password — do not commit it to Git or share it in screenshots.

4.2 Save the token on the PC

Open Command Prompt, PowerShell, or Terminal and run:

```
ngrok config add-authtoken <YOUR_NGROK_AUTHTOKEN>
```

Replace <YOUR_NGROK_AUTHTOKEN> with the value from the dashboard. The token is written to the default ngrok configuration file (ngrok.yml). You only have to run this command once unless you reset the token.

Optional check — open the config file:

```
ngrok config check
ngrok config edit
```

> **Security**
> If a token is leaked, go to the dashboard, reset the Authtoken, then run ngrok config add-authtoken again with the new value.

5. Create the HTTP forwarding tunnel

n8n listens on port 5678 by default. Point the tunnel at that port (class slides that show port 5000 are a generic Python example — change 5000 to 5678 for n8n).

5.1 Start the tunnel

In Command Prompt or Terminal:

```
ngrok http http://localhost:5678
```

Equivalent shorter form:

```
ngrok http 5678
```

5.2 Read the session screen

When the session is online you should see output similar to the following (values will differ):

```
Session Status     online
Account            you@example.com (Plan: Free)
Region             Asia Pacific (ap)
Web Interface      http://127.0.0.1:4040
Forwarding         https://xxxxxxxx.ngrok-free.app
                   -> http://localhost:5678
```

The value that matters is the HTTPS Forwarding URL, for example:

```
https://xxxxxxxx.ngrok-free.app
```

You can also copy the same URL from the dashboard: **Universal Gateway** → **Endpoints**. That page lists every live endpoint for your account.

5.3 Leave this window running

The tunnel exists only while this process is running. Do not close the ngrok window until you have finished testing webhooks.

> **Free-plan browser interstitial**
> The first time a browser hits a free ngrok URL, ngrok may show a warning page. API clients such as Telegram usually send a custom header and skip that page. If a browser test fails, retry or add the ngrok-skip-browser-warning header.

6. Set the WEBHOOK_URL environment variable for n8n

n8n builds production webhook URLs from WEBHOOK_URL. If this variable is missing, n8n advertises http://localhost:5678/... which Telegram and other services cannot call.

Use the HTTPS forwarding URL from the previous step. Do not append a path. A trailing slash is optional; be consistent.

6.1 Windows — current PowerShell session (temporary)

This lasts only for the current PowerShell window. Set it, then start n8n from the same window.

```
$env:WEBHOOK_URL = "https://xxxxxxxx.ngrok-free.app"

# confirm
echo $env:WEBHOOK_URL
```

6.2 Windows — permanent user variable

Use setx if you want the mapping to survive new terminals. Open a new PowerShell after running setx — existing windows do not pick up the change.

```
setx WEBHOOK_URL "https://xxxxxxxx.ngrok-free.app"
```

Verify in a brand-new PowerShell:

```
echo $env:WEBHOOK_URL
```

> **When the ngrok URL changes**
> Free-plan hostnames change on each new tunnel. After you start ngrok again, update WEBHOOK_URL to the new hostname (session variable or setx) and restart n8n. Stale WEBHOOK_URL values are the most common reason Telegram webhooks point at a dead address.

6.3 macOS and Linux

For the current terminal session:

```
export WEBHOOK_URL="https://xxxxxxxx.ngrok-free.app"
echo $WEBHOOK_URL
```

To make it persist, add the export line to ~/.bashrc, ~/.zshrc, or a dedicated env file that you source before starting n8n. Restart the shell after editing.

6.4 Recommended startup order

1. Start ngrok and wait until Session Status is online.
2. Copy the HTTPS Forwarding URL.
3. Set WEBHOOK_URL in the same terminal that will launch n8n (or confirm the permanent variable is current).
4. Start n8n.
5. Activate the workflow so n8n registers the webhook with Telegram or the other provider.

7. Launch n8n

7.1 Windows

From the same PowerShell where WEBHOOK_URL is set:

```
n8n.cmd start
```

If n8n was installed globally via npm:

```
n8n start
```

7.2 macOS and Linux

```
n8n start
```

The editor is normally at http://localhost:5678. Open that address in a browser on the same machine. External callers should use the ngrok HTTPS URL, not localhost.

Confirm n8n saw the variable

In a Webhook node or Telegram Trigger, inspect the Production URL. It must begin with your ngrok hostname (https://xxxxxxxx.ngrok-free.app/webhook/... or /webhook-test/...), not http://localhost:5678.

If it still shows localhost, n8n was started before WEBHOOK_URL was set. Stop n8n, set the variable, and start it again.

8. Verify the webhook (Telegram example)

Class three uses a Telegram Trigger as the inbound webhook. After the workflow is active, confirm that Telegram stored the public URL.

In a browser or with curl, call:

```
https://api.telegram.org/bot<YOUR_BOT_TOKEN>/getWebhookInfo
```

A healthy response looks like:

```
{"ok":true,"result":{"url":"https://xxxxxxxx.ngrok-free.app/webhook/....",
"has_custom_certificate":false, ... }}
```

The url field must match your current ngrok hostname plus the path n8n generated. If url is empty, the workflow is not active or WEBHOOK_URL was wrong when n8n started. If url points at an old ngrok hostname, restart the tunnel, update WEBHOOK_URL, restart n8n, and toggle the workflow off/on so Telegram is re-registered.

You can also watch live requests in the ngrok inspector at http://127.0.0.1:4040 (or the Web Interface port printed in the ngrok window). Successful POSTs appear there as they arrive.

9. End-to-end command checklist

Substitute your real token and the hostname printed by your live ngrok session.

Windows (Command Prompt + PowerShell)

```
ngrok config add-authtoken <YOUR_NGROK_AUTHTOKEN>
ngrok http http://localhost:5678

# in a second PowerShell window, after copying the HTTPS URL:
$env:WEBHOOK_URL = "https://xxxxxxxx.ngrok-free.app"
echo $env:WEBHOOK_URL
n8n.cmd start
```

macOS / Linux

```
ngrok config add-authtoken <YOUR_NGROK_AUTHTOKEN>
ngrok http http://localhost:5678

# in a second terminal:
export WEBHOOK_URL="https://xxxxxxxx.ngrok-free.app"
echo $WEBHOOK_URL
n8n start
```

10. Common problems

Symptom	What to check
ERR_NGROK_108 / failed to auth	Authtoken missing or reset. Run ngrok config add-authtoken again.
Tunnel online but n8n URLs still show localhost	WEBHOOK_URL was not visible to the n8n process. Set it in the same shell, then restart n8n.
Telegram works once, then dies after reboot	Free ngrok URL changed. Start a new tunnel, update WEBHOOK_URL, restart n8n, reactivate the workflow.
getWebhookInfo url is empty	Workflow is inactive, or the Telegram credential / bot token is wrong.
502 / connection refused in ngrok inspector	n8n is not running, or ngrok is forwarding the wrong port. Confirm port 5678.
Wrong port forwarded	Class Python demos use 5000. For n8n always use 5678 unless you changed N8N_PORT.

11. Quick reference — values used in this lab

Item	Value / command
Local n8n URL	`http://localhost:5678`
Tunnel command	`ngrok http http://localhost:5678`
Auth command (once)	`ngrok config add-authtoken <token>`
Windows session env	`$env:WEBHOOK_URL = "https://<subdomain>.ngrok-free.app"`
Windows permanent env	`setx WEBHOOK_URL "https://<subdomain>.ngrok-free.app"`
macOS / Linux env	`export WEBHOOK_URL="https://<subdomain>.ngrok-free.app"`
Start n8n (Windows)	`n8n.cmd start`
Telegram check	`https://api.telegram.org/bot<TOKEN>/getWebhookInfo`
ngrok request inspector	`http://127.0.0.1:4040`

Sources for this procedure: Class three slides (Setup of Web Tunneling and Web Hooking) and the attached webhooking checklist. Replace every sample hostname with the live URL printed by your own ngrok session.